

LAPORAN TUGAS
DINING PHILOSOPHERS PROBLEM



Dosen Pengampu : Triawan Adi Cahyanto, M.K

Di susun Oleh:

Muhammad maulana wiramatajaya 2410651006

PROGRAM STUDY TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2024

DAFTAR ISI

BAB I	2
PEMBAHASAN	3
1.1. Analisis Masalah	3
1.2. Implementasi deadlock	4
A. Flowchart	4
B. Penjelasan kode	5
1.3. Testing & Output	7
A. Output	7
Deadlock	Error! Bookmark not defined.
B. Analisis Hasil	8
BAB II	10
Kesimpulan	Error! Bookmark not defined.

PEMBAHASAN

1.1. Analisis Masalah

1.1.1. Pemahaman Soal

Terjadi Deadlock (Kebuntuan) Dikarenakan kondisi di mana dua atau lebih proses (dalam hal ini, filsuf) berada dalam keadaan menunggu tak terbatas untuk sumber daya (dalam hal ini, sumpit) yang dipegang oleh proses lain dalam siklus menunggu.

1.1.2. Solusi Masalah Deadlock

Dalam kasus di atas ada beberapa Solusi, akan tetapi kami hanya menggunakan 2 solusi sajaya yaitu :

- Resource Ordering : filsuf genap ambil kiri dulu, ganjil ambil kanan dulu.
- Limiting Philosophers : max 4 filsuf makan bersamaan.

1.1.3. Solusi yang paling adil

solusi Resource Ordering (Pencegahan Circular Wait) adalah solusi yang paling sederhana dan efektif untuk menjamin keadilan dasar, karena memastikan bahwa setiap filsuf (proses) pada akhirnya akan mendapatkan kesempatan untuk makan, sehingga menghindari *deadlock* dan *starvation* (asumsi implementasi tidak bias).

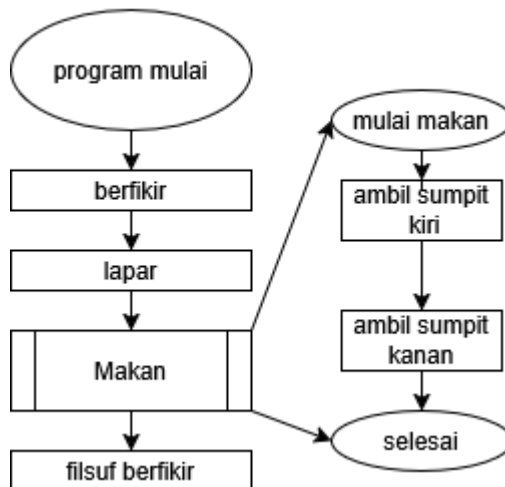
1.1.4. kemungkinan starvation

Starvation terjadi ketika suatu proses (filsuf) terus-menerus ditolak aksesnya ke sumber daya yang dibutuhkan (sumpit) karena proses lain (filsuf lain) selalu berhasil mengambil sumber daya tersebut terlebih dahulu.

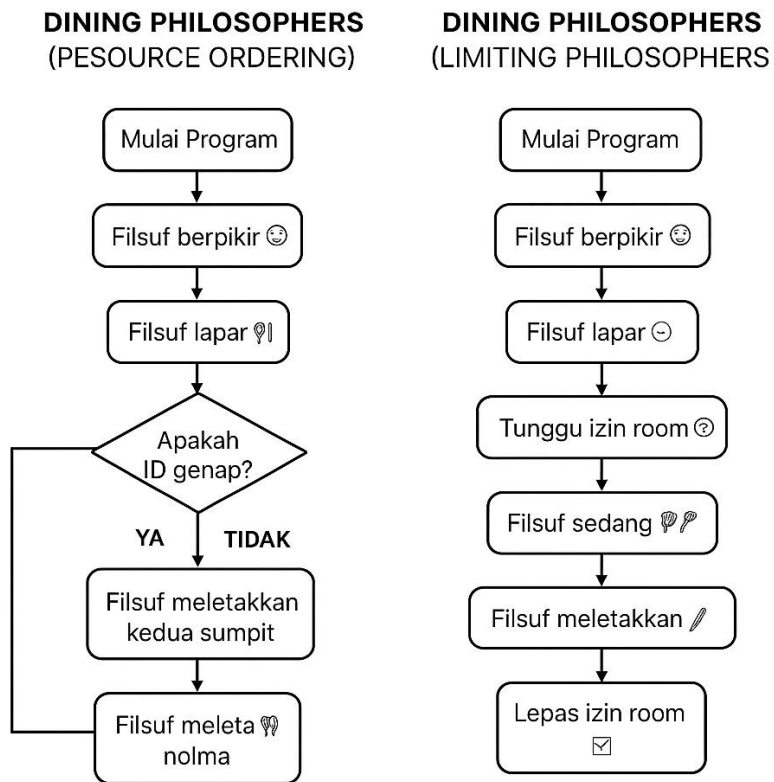
1.2. Implementasi Deadlock

1.2.1. Flowchart

A. Deadlock



A. Solusi Deadlock



1.2.2. Penjelasan kode

A. DeadLock

```
#define N 5 // jumlah filsuf
#define THINKING 0
#define HUNGRY 1
#define EATING 2

pthread_mutex_t chopstick[N];

void* philosopher(void* num) {
    int id = *(int*)num;
    while (1) {
        printf("🧐 Filsuf-%d sedang berpikir...\n", id);
        sleep(1);
        printf("🍴 Filsuf-%d lapar dan mencoba mengambil sumpit...\n", id);
        // DEADLOCK VERSION: ambil sumpit kiri dulu, lalu kanan
        pthread_mutex_lock(&chopstick[id]); // sumpit kiri
        printf("🍴 Filsuf-%d mengambil sumpit kiri (%d)\n", id, id);
        sleep(1);
        pthread_mutex_lock(&chopstick[(id+1)%N]); // sumpit kanan
        printf("🍴 Filsuf-%d mengambil sumpit kanan (%d)\n", id, (id+1)%N);
        printf("🍽️ Filsuf-%d sedang MAKAN...\n", id);
        sleep(2);
        pthread_mutex_unlock(&chopstick[id]);
        pthread_mutex_unlock(&chopstick[(id+1)%N]);
        printf("👋 Filsuf-%d selesai makan, meletakkan sumpit.\n", id);
    }
}

int main() {
    pthread_t tid[N];
    int i, phil_id[N];

    for (i=1; i<N; i++)
        pthread_mutex_init(&chopstick[i], NULL);

    printf("=== DINING PHILOSOPHERS (DEADLOCK VERSION) ===\n");
    for (i=1; i<N; i++) {
        phil_id[i] = i;
        pthread_create(&tid[i], NULL, philosopher, &phil_id[i]);
    }

    for (i=1; i<N; i++)
        pthread_join(tid[i], NULL);
}
```

Solusi 1

Pesource Ordering

```
5 #define N 5
6 pthread_mutex_t chopstick[N];
7
8 void* philosopher(void* num) {
9     int id = *(int*)num;
10    while (1) {
11        printf("😴 Filsuf-%d berpikir...\n", id);
12        sleep(1);
13        printf("🍴 Filsuf-%d lapar...\n", id);
14
15        if (id % 2 == 0) { // genap: kiri dulu
16            pthread_mutex_lock(&chopstick[id]);
17            pthread_mutex_lock(&chopstick[(id+1)%N]);
18        } else { // ganjil: kanan dulu
19            pthread_mutex_lock(&chopstick[(id+1)%N]);
20            pthread_mutex_lock(&chopstick[id]);
21        }
22
23        printf("🍽️ Filsuf-%d makan...\n", id);
24        sleep(2);
25        pthread_mutex_unlock(&chopstick[id]);
26        pthread_mutex_unlock(&chopstick[(id+1)%N]);
27        printf("🍀 Filsuf-%d selesai makan.\n", id);
28    }
29 }
30
31 int main() {
32     pthread_t tid[N];
33     int phil_id[N];
34
35     for (int i=0; i<N; i++)
36         pthread_mutex_init(&chopstick[i], NULL);
37
38     printf("=== DINING PHILOSOPHERS (RESOURCE ORDERING) ===\n");
39     for (int i=0; i<N; i++) {
40         phil_id[i] = i;
41         pthread_create(&tid[i], NULL, philosopher, &phil_id[i]);
42     }
43     for (int i=0; i<N; i++)
44         pthread_join(tid[i], NULL);
45 }
```

Solution

Limiting philosopher

```
#define N 5
pthread_mutex_t chopstick[N];
sem_t room; // membatasi jumlah filsuf

void* philosopher(void* num) {
    int id = *(int*)num;
    while (1) {
        printf("😴 Filsuf-%d berpikir...\n", id);
        sleep(1);
        printf("🍴 Filsuf-%d lapar...\n", id);
        sem_wait(&room); // masuk ruangan (maks 4)
        pthread_mutex_lock(&chopstick[id]);
        pthread_mutex_lock(&chopstick[(id+1)%N]);
        printf("🍽️ Filsuf-%d makan...\n", id);
        sleep(2);
        pthread_mutex_unlock(&chopstick[id]);
        pthread_mutex_unlock(&chopstick[(id+1)%N]);
        sem_post(&room); // keluar ruangan
        printf("🍀 Filsuf-%d selesai makan.\n", id);
    }
}

int main() {
    pthread_t tid[N];
    int phil_id[N];

    for (int i=1; i<=N; i++)
        pthread_mutex_init(&chopstick[i], NULL);

    sem_init(&room, 1, 4); // maksimal 4 filsuf makan bersamaan

    printf("=== DINING PHILOSOPHERS (LIMITING PHILOSOPHERS) ===\n");
    for (int i=1; i<=N; i++) {
        phil_id[i] = i;
        pthread_create(&tid[i], NULL, philosopher, &phil_id[i]);
    }

    for (int i=1; i<=N; i++)
        pthread_join(tid[i], NULL);
}
```

1.3. Testing & Output

1.3.1. Output

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_5/tugas$ ./deadlock
=== DINING PHILOSOPHERS (DEADLOCK VERSION) ===
🤖 Filsuf-2 sedang berpikir...
🤖 Filsuf-1 sedang berpikir...
🤖 Filsuf-3 sedang berpikir...
🤖 Filsuf-4 sedang berpikir...
🤖 Filsuf-1 lapar dan mencoba mengambil sumpit...
🤖 Filsuf-2 lapar dan mencoba mengambil sumpit...
🤖 Filsuf-2 mengambil sumpit kiri (2)
🤖 Filsuf-1 mengambil sumpit kiri (1)
🤖 Filsuf-3 lapar dan mencoba mengambil sumpit...
🤖 Filsuf-3 mengambil sumpit kiri (3)
🤖 Filsuf-4 lapar dan mencoba mengambil sumpit...
🤖 Filsuf-4 mengambil sumpit kiri (4)
🤖 Filsuf-4 mengambil sumpit kanan (0)
🤖 Filsuf-4 sedang MAKAN...
🤖 Filsuf-4 selesai makan, meletakkan sumpit.
🤖 Filsuf-4 sedang berpikir...
🤖 Filsuf-3 mengambil sumpit kanan (4)
🤖 Filsuf-3 sedang MAKAN...
🤖 Filsuf-4 lapar dan mencoba mengambil sumpit...
🤖 Filsuf-3 selesai makan, meletakkan sumpit.
🤖 Filsuf-3 sedang berpikir...
🤖 Filsuf-2 mengambil sumpit kanan (3)
🤖 Filsuf-2 sedang MAKAN...
🤖 Filsuf-4 mengambil sumpit kiri (4)
🤖 Filsuf-3 lapar dan mencoba mengambil sumpit...
🤖 Filsuf-4 mengambil sumpit kanan (0)
🤖 Filsuf-4 sedang MAKAN...
🤖 Filsuf-2 selesai makan, meletakkan sumpit.
🤖 Filsuf-2 sedang berpikir...
🤖 Filsuf-3 mengambil sumpit kiri (3)
🤖 Filsuf-1 mengambil sumpit kanan (2)
🤖 Filsuf-1 sedang MAKAN...
🤖 Filsuf-4 selesai makan, meletakkan sumpit.
🤖 Filsuf-4 sedang berpikir...
🤖 Filsuf-2 lapar dan mencoba mengambil sumpit...
🤖 Filsuf-3 mengambil sumpit kanan (4)
🤖 Filsuf-3 sedang MAKAN...
```

Solusi 1

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_5/tugas$ gcc -o solusi
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_5/tugas$ ./solutionA
=== DINING PHILOSOPHERS (RESOURCE ORDERING) ===
🤖 Filsuf-0 berpikir...
🤖 Filsuf-1 berpikir...
🤖 Filsuf-2 berpikir...
🤖 Filsuf-3 berpikir...
🤖 Filsuf-4 berpikir...
🤖 Filsuf-0 lapar...
🤖 Filsuf-0 makan...
🤖 Filsuf-2 lapar...
🤖 Filsuf-2 makan...
🤖 Filsuf-1 lapar...
🤖 Filsuf-4 lapar...
🤖 Filsuf-3 lapar...
🤖 Filsuf-0 selesai makan.
🤖 Filsuf-0 berpikir...
🤖 Filsuf-4 makan...
🤖 Filsuf-2 selesai makan.
🤖 Filsuf-2 berpikir...
🤖 Filsuf-1 makan...
🤖 Filsuf-0 lapar...
🤖 Filsuf-2 lapar...
🤖 Filsuf-4 selesai makan.
🤖 Filsuf-4 berpikir...
🤖 Filsuf-3 makan...
🤖 Filsuf-0 makan...
🤖 Filsuf-1 selesai makan.
🤖 Filsuf-1 berpikir...
🤖 Filsuf-4 lapar...
🤖 Filsuf-1 lapar...
🤖 Filsuf-3 selesai makan.
🤖 Filsuf-3 berpikir...
🤖 Filsuf-0 selesai makan.
🤖 Filsuf-0 berpikir...
```

Solusi 2

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_5/tugas$ gcc -o solusi
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_5/tugas$ ./solutionB
=== DINING PHILOSOPHERS (LIMITING PHILOSOPHERS) ===
🤖 Filsuf-0 berpikir...
🤖 Filsuf-2 berpikir...
🤖 Filsuf-1 berpikir...
🤖 Filsuf-4 berpikir...
🤖 Filsuf-3 berpikir...
🤖 Filsuf-0 lapar...
🤖 Filsuf-0 makan...
🤖 Filsuf-2 lapar...
🤖 Filsuf-2 makan...
🤖 Filsuf-1 lapar...
🤖 Filsuf-4 lapar...
🤖 Filsuf-3 lapar...
🤖 Filsuf-4 makan...
🤖 Filsuf-0 selesai makan.
🤖 Filsuf-0 berpikir...
🤖 Filsuf-2 selesai makan.
🤖 Filsuf-2 berpikir...
🤖 Filsuf-1 makan...
🤖 Filsuf-0 lapar...
🤖 Filsuf-2 lapar...
🤖 Filsuf-4 selesai makan.
🤖 Filsuf-4 berpikir...
🤖 Filsuf-3 makan...
🤖 Filsuf-1 selesai makan.
🤖 Filsuf-1 berpikir...
🤖 Filsuf-0 makan...
^C
```

1.3.2. Analisis Hasil

1. Versi Deadlock (Semua filsuf ambil sumpit kiri dulu)

Hasil pengujian:

Pada saat program dijalankan, setiap filsuf (thread) menampilkan status berpikir → lapar → mengambil sumpit kiri → mencoba mengambil sumpit kanan. Setelah beberapa detik, seluruh filsuf berhenti di kondisi “mengambil sumpit kanan”. Tidak ada lagi aktivitas di layar dan program berhenti merespons.

Analisis:

- Kondisi ini menunjukkan terjadinya deadlock, karena setiap filsuf sudah memegang satu sumpit (kiri) dan menunggu sumpit kanan yang dipegang filsuf sebelahnya.
- Terjadi circular wait, yaitu setiap thread menunggu resource yang dipegang thread lain, membentuk lingkaran menunggu.
- Tidak ada mekanisme untuk membatalkan atau mengubah urutan pengambilan resource, sehingga tidak ada yang bisa melanjutkan eksekusi.
- Hasil ini sesuai dengan teori bahwa Dining Philosophers Problem menggambarkan situasi deadlock murni.

Kesimpulan :

Program tidak berjalan dengan baik karena semua filsuf terjebak dalam keadaan menunggu selamanya.

2. Solusi A – Resource Ordering (Even/Odd Rule)

Hasil pengujian:

Program berjalan stabil tanpa hang. Semua filsuf dapat bergantian makan dan berpikir. Pada output, terlihat bahwa tidak ada filsuf yang menunggu terlalu lama, dan semua mendapatkan giliran makan secara adil.

Analisis:

- Strategi ini mengubah urutan pengambilan resource agar tidak semua filsuf melakukan langkah yang sama.
- Filsuf genap mengambil sumpit kiri dulu, sedangkan ganjil mengambil sumpit kanan dulu.
- Dengan aturan ini, tidak ada lagi circular wait, karena setidaknya satu filsuf dapat mengambil dua sumpit dan menyelesaikan makannya.
- Deadlock berhasil dihindari karena kondisi deadlock (mutual exclusion, hold & wait, no preemption, circular wait) tidak lagi terpenuhi.
- Efisiensi cukup tinggi karena semua filsuf dapat bergiliran makan dengan keadilan (fairness baik).

Kesimpulan :

Deadlock berhasil dicegah.

Distribusi makan adil dan program stabil.

Cocok untuk sistem yang ingin menjaga keadilan tanpa membatasi jumlah thread aktif.

3. Solusi B – Limiting Philosophers (Semaphore “room”)

Hasil pengujian:

Program juga berjalan lancar. Tidak terjadi deadlock maupun starvation. Filsuf masuk ke ruang makan maksimal empat orang, sedangkan satu filsuf selalu menunggu di luar sampai ada yang selesai makan.

Analisis:

- Dengan semaphore room, sistem membatasi maksimal empat filsuf yang boleh memegang sumpit pada satu waktu.
- Dengan begitu, selalu ada minimal satu sumpit bebas, mencegah terjadinya circular wait.
- Solusi ini efektif untuk mencegah deadlock pada sistem dengan jumlah resource terbatas.
- Tingkat efisiensi sedikit di bawah resource ordering, karena ada waktu tunggu tambahan untuk filsuf yang menunggu izin masuk ruangan.

Kesimpulan :

Deadlock tidak terjadi.

Starvation tidak terdeteksi.

Sistem stabil, meski sedikit kurang efisien karena pembatasan jumlah filsuf yang aktif.

KESIMPULAN

Dari uraian yang telah dibahas sebelumnya maka dalam tugas akhir ini dapat disimpulkan bahwa dengan menggunakan teori analisa simulasi maka akan mempermudah dalam pemahaman permasalahan yang ada. Seperti algoritma Dining Philosophers Problem dalam sistem operasi ini, dapat diamati tingkah laku dari filsuf (sedang makan, berfikir atau lapar) dengan banyak filsuf minimum 2 dan maksimum 5 dan lama proses maksimum adalah 99 satuan waktu. Bahwa dengan pemberian variabel masukan waktu tunggu yang terlalu tinggi pada filsuf akan mengakibatkan sebagian besar waktu filsuf berada dalam periode lapar. Sedangkan dengan waktu tunggu filsuf yang kecil akan mengakibatkan lama lapar filsuf juga kecil dan filsuf akan sering melakukan aktivitas makan dan berfikir. Dalam simulasi ini variabel masukan waktu tunggu adalah lebih kecil atau sama dengan lama proses.

DAFTAR PUSTAKA

https://repository.uin-suska.ac.id/664/1/2011_2011170-.pdf

<https://www.bajangjournal.com/index.php/JIRK/article/view/6711/5156>